EAST WEST INSTITUTE OF TECHNOLOGY

Sy. No.63,Off. Magadi Road, Vishwaneedam Post, Bengaluru-560091

(Autonomous Institute under Visvesvaraya Technological University,Belagavi)

DEPARTMENT OF INFORMATION SCIENCEAND ENGINEERING

BIG DATA ANALYTICS LAB MANUAL

Mrs.AKSHATHA K S
Asst Prof . Dept of ISE, EWIT

SLNO	EXPERIMENTS
1	Install Hadoop and implement the following management task in Hadoop. Adding files and directories retrieve files deleting files and directories
2	Develop a Map Reduce program to implement matrix multiplication
3	Develop a Map Reduce program that mines weather data and display's appropriate message indicating the weather condition of the day
4	Develop a Map Reduce program to find the tags associated with each movie by Analyzing movies lens data
5	Implement functions count,sort,limit,skip,aggregate using MongoDB
6	Develop Pig ,Latin script to sort,group,join,project,and filter the data
7	Use Hive to create,alter and drop database,table,views,functions and indexes
8	Implement a word count program in Hadoop and spark
9	Use CDH(cloudierdistributionforHadoop) and HVE(Hadoop user interface)to analyse data and generate report for sample dataset

Program1: Install Hadoop and implement the following management task in Hadoop

Adding files and directories retrieve files deleting files and directories.

Update system

Sudo apt update

sudo apt upgrade-y

Install Java(HadoopneedsJava)

sudo apt install openjdk-8-jdk-y

Verify Java:

java-version

Create HadoopUser(Recommended)

sudo adduser hadoop

Sudo usermod-aGsudo hadoop

Switch to Hadoop user:

su- hadoop

Download and Install Hadoop

wget

https://downloads.apache.org/hadoop/common/

hadoop-3.3.6/hadoop-3.3.6.tar.gz

Extract:

tar-xvzfhadoop-3.3.6.tar.gz

mv hadoop-3.3.6 hadoop

Set Hadoop Environment Variables

Edit .bashrc:

nano ~/.bashrc

Addattheend:

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-

amd64

exportHADOOP_HOME=/home/hadoop/hadoop

export

PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_H

OME/sbin

Applychanges:

source~/.bashrc

exportHADOOP_HOME=/home/ewit/hadoop Step 1 — Set Correct HADOOP_HOME

Run this:

Export HADOOP_HOME=/home/ewit/hadoop

Step 2 — Add to PATH

export

PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_H

OME/sbin

Step3 —Check

echo$ HADOOP_HOME

It should show:

/home/ewit/hadoop

✅Now Go to Configuration Folder

cd $HADOOP_HOME/etc/hadoop

ls

Now it should work perfectly ✅

You will see:

core-site.xml

hdfs-site.xml

mapred-site.xml

Configure Hadoop(Pseudo-DistributedMode)

Editcore-site.xml

Nano $HADOOP_HOME/etc/hadoop/core-site.xml

```
<configuration>
 <property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
 </property>
</configuration>
```

Edithdfs-site.xml

nano$HADOOP_HOME/etc/hadoop/hdfs-site.xml

```
<configuration>
 <property>
  <name>dfs.replication</name>
  <value>1</value>
 </property>
```

</configuration>

Format Name Node(OneTimeOnly)

hdfs namenode -format

STEP1:Install SSH Server

Switch back to your normal user(ewit):

exit

Then install SSH:

Sudo apt update

Sudo apt install open ssh-server

STEP2:Start SSH Service

After installation, run:

Sudo service ssh start

Check if running:

Sudo service ssh status

You should see:

Active:active(running)

Now switch back:

su- hadoop

ssh-keygen-trsa

cat~/.ssh/id_rsa.pub>>~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys

Test: ssh local host

Ifitlogsinwithoutaskingpassword→SUCCESS

✅

exit

As hadoop user,run:

java-version

open jdk version"11..."

If not installed,switch to normal user and install:

exit

Sudo apt install openjdk-11-jdk

Then go back:

su- hadoop

Find Java Path

readlink-f$(whichjava)

/usr/lib/jvm/java-11-openjdk-amd64/bin/java

/usr/lib/jvm/java-11-openjdk-amd64

Set JAVA_HOME in Hadoop

Nano$HADOOP_HOME/etc/hadoop/hadoop-

env.sh

export JAVA_HOME=/usr/lib/jvm/java-11-

open jdk-amd64

Reload and StartHadoop

source ~/.bashrc

start-dfs.sh

Reload and s tart Hadoop

PARTB:Hadoop File Management Tasks(HDFS)

Adding Files and Directories

Create directory in HDFS

hdfs dfs -mkdir /user

hdfsdfs-mkdir/user/hadoop

hdfsdfs-mkdir/user/hadoop/input

Create a local file

nano sample.txt

Add:

Welcome to Hadoop HDFS

File management example

Copy file from local system to HDFS

hdfsdfs-putsample.txt/user/hadoop/input Verify:

hdfsdfs-ls/user/hadoop/input

Retrieving Files from HDFS

Copy file from HDFS to localsystem

Hdf sdfs-get/user/hadoop/input/sample.txt

View file content directly

hdfsdfs-cat/user/hadoop/input/sample.txt

Deleting Files and

Directories Delete a file

hdfsdfs-rm/user/hadoop/input/sample.txt

Delete an empty directory

hdfsdfs-rmdir/user/hadoop/input

Delete a directory with files(recursive)

hdfs dfs -rm -r /user/hadoop

Program2: Develop a Map Reduce program to implement matrix multiplication.

To create a mapper file

nano mapper.py

Python Code:

```
#!/usr/bin/env python3
import sys

# Define matrix dimensions (should be adapted to actual input)
A_ROWS = 2  # Number of rows in Matrix A
B_COLS = 2  # Number of columns in Matrix B

for line in sys.stdin:
        line = line.strip()
        matrix, row, col, value = line.split()
        row, col, value = int(row), int(col), int(value)

        if matrix == "A":
# Emit for every column in B
for k in range(B_COLS):
        print(f"{row} {k}\tA {col} {value}")

elif matrix == "B":
# Emit for every row in A
        for i in range(A_ROWS):
                print(f"{i} {col}\tB {row} {value}")
```

To create a reducer file

nano reducer.py

Python Code-

```
import sys
from collections import defaultdict

# Dictionary to store intermediate values
intermediate = defaultdict(list)

for line in sys.stdin:
        line = line.strip()
        key, value = line.split("\t")
i, j = map(int, key.split())
        parts = value.split()

intermediate[(i, j)].append((parts[0], int(parts[1]), int(parts[2])))
```

```
# Compute final matrix multiplication
for (i, j), values in intermediate.items():
a_values = {k: v for m, k, v in values if m == "A"}
b_values = {k: v for m, k, v in values if m == "B"}

    result = sum(a_values[k] * b_values[k] for k in a_values if k in b_values)

     if result:
       print(f"{i} {j} {result}")
```

To create input file

nano matrix.txt

```
A 0 0 3
A 0 1 2
A 1 0 1
A 1 1 4
B 0 0 2
B 1 0 5
B 0 1 3
B 1 1 1
```

To make files executable:

```
Chmod +xmapper.py
chmod +x reducer.py
```

Command to run the program

```
cat matrix.txt | python3  mapper.py | sort | python3 reducer.py
```

Output:

```
0 0 16
0 1 11
1 0 22
1 1 7
```

Program3: Develop a Map Reduce program that mines weather data and displays appropriate message indicating the weather condition of the day.

Sample Input Dataset

Create input file:nano weather.txt

2025-02-01,Mangalore,32,40

202502-01,Udupi,28,70

2025-02-01,Bangalore,18,80

2025-02-02,Mangalore,35,30

2025-02-02,Udupi,22,55

Mapper code:

```
#!/usr/bin/env
python3 import sys
 for line in sys.stdin:
    line=line.strip()

if not line:
    continue

date,city,temp,hum=line.split(",")
temp=float(temp)
hum = float(hum)


if temp>=30 and hum<50:
   condition = "Hot Day"
elif
temp< 20 and hum>=70:
    condition = "Cold Day"
elif 25<=temp <30 and hum>=60:
     condition = "Rainy Day"

else:
    condition="PleasantDay"

print(f"{date}\t{city}\t{condition}")
```

Reducer code

```
#!/usr/bin/envpython3

  import sys

  for line in sys.stdin:

        line=line.strip()

        date,city,condition=line.split("\t")

        print(f"{date} {city} -> {condition}")
```

To make files executable

```
chmod +x mapper.py

 chmod+x  reducer.py
```

To run the Program

```
cat weather.txt|python3 mapper.py|sort|python3 reducer.py
```

Output:

```
2025-02-02,Mangalore->Rainy day
```

Program4: Develop a Map Reduce program to find the tags associated with each movie by analysing movies lens data.

Map Reduce Program to Find Movie

Tags(Hadoop Streaming)

To create input file:

nano movie_tags.txt

1,Action

1,Thriller

2,Comedy

2,Romance

3,Sci-Fi

3,Adventure

1,Drama

Mapper Code:Mapper.py

```
import sys

for line in sys.stdin:
     line = line.strip()

   try:
      # Split assuming exactly 2 elements, or adjust if you have more columns
        movie_id, tag = line.split(",")
        movie_id = movie_id.strip()
        tag = tag.strip()

      # Skip the header row if it appears in the data
      if  movie_id == "userId":
              continue

      print(f"{movie_id}\t{tag}")

   except ValueError:
      # This safely skips lines that don't have exactly 2 columns
      continue
```

Reducer code:Reducer.py

```
import sys
from collections import defaultdict

movie_tags = defaultdict(list)

for line in sys.stdin:

        line = line.strip()

if  not line:  # Safe check to skip empty lines
      continue


 try:
      # Move these two lines INSIDE the for loop (indented)
      movie_id, tag = line.split("\t", 1)
      movie_tags[movie_id].append(tag)
except ValueError:
      continue  # Skips lines that don't have a tab separator

# Keep this loop OUTSIDE the first loop to print the final results
for movie_id, tags in movie_tags.items():
    print(f"{movie_id}\t{', '.join(tags)}")
```

Command to run the program

cat movies.txt|python3 mapper.py|sort|python3 reducer.py

Output

```
"1"   ["Action","Thriller","Drama"]
"2"   ["Comedy","Romance"]
"3"   ["Sci-Fi","Adventure"]
```

Program5: Implement functions count,sort,limit,skip,aggregate using MongoDB.

Mongo DB mongosh Shell Guide with Student Example Program with

step by step

This document show how students can use the MongoDBShell(mongosh) in Linux and execute MongoDB operations such as Count, Sort, Limit, Skip, and Aggregate using a Student Marks database example.

1. Install Mongo DB(Linux)

Update packages:

sudo apt update Install MongoDB:

sudo apt install mongodb

2. Start MongoDB Service

Start the MongoDB server:

sudo systemctl start mongodb

or

sudo systemctl start mongod

Check server status:

sudo systemctl status mongod

or

sudo systemctl status mongodb

3. Open MongoDB Shell (mongosh):

Open the MongoDB shell by typing: mongosh

If successful, the shell prompt will appear.

Or

Type mongo for older versions

4. **Create or Switch Database**

Create and switch to a database name

dstudentDB:

Type:

use studentDB

5. **Insert Student Data**

Insert sample student records into the collection:

```
db.students.insertMany([
{name:"Asha",age:20,course:"CSE",marks:85,city:"Bangalore"},
{name:"Rahul",age:22,course:"ECE",marks:72,city:"Delhi"},
{name:"Priya",age:21,course:"CSE",marks:91,city:"Mumbai"},
{name:"Kiran",age:23,course:"EEE",marks:65,city:"Chennai"},
{name:"Sneha",age:20,course:"CSE",marks:88,city:"Hyderabad"},
{name:"Arjun",age:22,course:"ECE",marks:76,city:"Pune"},
{name:"Divya",age:21,course:"CSE",marks:95,city:"Bangalore"},
{name:"Manoj",age:24,course:"EEE",marks:60,city:"Delhi"}
])
```

To check data is inserted or not:

db.students.find()

Output:

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b08"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b09"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0a"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0b"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }

{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0c"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }

{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0d"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }

{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0e"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }

{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0f"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }

6. **Count Function**

To Count total number of students

```
db.students.countDocuments()
```

or

```
db.students.count()
```

Output:8

To count students in CSE Department:

```
db.students.countDocuments({course:"CSE"})
```

or

```
db.students.count({course:"CSE"}
```

Output:4

7. **Sort Function**:To Sort students by marks and age in ascending and descending order

a:To Sort students by marks in descending order:

```
db.students.find().sort({marks:-1})
```

Output:

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0a"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0e"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b08"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0c"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b09"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0d"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0b"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0f"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }

b: To Sort students by age in ascending order:

```
db.students.find().sort({age:1})
```

Output:

{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b08"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0c"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0a"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0e"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b09"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0d"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0b"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0f"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }

8. Limit Function

Display top 3 students based on marks:

```
db.students.find().sort({marks:-1}).limit(3)
```

Output:

```
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0a"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0e"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b08"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
```

9. Skip Function

a:Skip first two records:

```
db.students.find().skip(2)
```

Output:

```
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0a"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0b"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0c"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0d"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" : 72, "city" : "Delhi" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0e"), "name" : "Priya", "age" : 21, "course" : "CSE", "marks" : 91, "city" : "Mumbai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0f"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }
```

b:To skip and limit the records per page:

We use Pagination example (Page 2):

```
db.students.find().skip(3).limit(3)
```

Output:

```
{ "_id" : ObjectId("6a6ad503c4f4ce4b0b8b1b0b"), "name" : "Kiran", "age" : 23, "course" : "EEE", "marks" : 65, "city" : "Chennai" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0c"), "name" : "Asha", "age" : 20, "course" : "CSE", "marks" : 85, "city" : "Bangalore" }
{ "_id" : ObjectId("6a6ad53cc4f4ce4b0b8b1b0d"), "name" : "Rahul", "age" : 22, "course" : "ECE", "marks" :  72, "city" : "Delhi" }
```

10. Aggregate Function

a: Average marks of all students:

```
db.students.aggregate([
{$group:{_id:null,averageMarks:{$avg:"$marks"}}}
])
```

Output:

```
{$group:{_id:null,averageMarks:{$avg:"$marks"}}}
... ])
{ "_id" : null, "averageMarks" : 78.25 }
```

b: Average marks by course:

```
db.students.aggregate([
{
 $group:{
  _id:"$course",
  avgMarks:{$avg:"$marks"}
 }
}
])
```

Output:

```
{ "_id" : "EEE", "avgMarks" : 65 }
{ "_id" : "ECE", "avgMarks" : 72 }
{ "_id" : "CSE", "avgMarks" : 88 }
```

c: Highest marks in each course:

```
db.students.aggregate([
{
 $group
        _id:"$course",      highestMarks:{$max:"$marks"}
 }
}
])
```

Output:

```
{ "_id" : "EEE", "highestMarks" : 65 }
{ "_id" : "ECE", "highestMarks" : 72 }
{ "_id" : "CSE", "highestMarks" : 91
```

11. Exit MongoDBShell

To exit the mongosh shell:

```
exit
```

Result

Students successfully executed MongoDB operations such as Count, Sort, Limit, Skip, and Aggregate using the mongosh shell.

Program 6: Develop Pig Latin script to sort, group, join, project, and filter the data.
Guide (Local Mode)

Step 1: Install Java
Check Java version: java -version
Install if not present: sudo apt update
sudo apt install openjdk-8-jdk

Step 2: Download Apache Pig

wget https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz

Step 3: Extract Pig:
tar -xvf pig-0.17.0.tar.gz
cd pig-0.17.0

Step 4: Set Environment Variables
nano ~/.bashrc => Open this file

And add these 2 lines to set environment variables

export PIG_HOME=~/pig-0.17.0
export PATH=$PATH:$PIG_HOME/bin

source ~/.bashrc => we can open the file as above mentioned by using this command

Step 5: Create Project Directory

create folder for pig_project

mkdir pig_project

To navigate into pig_project folder

cd pig_project

Step 6: Create Input Files
nano employees.txt

=>Open the file and add the below data
1,Anu,25,HR,30000
2,Akshay,30,IT,50000
3,Bindu,28,IT,45000
4,Disha,35,HR,40000

nano dept.txt

=>Open the file and add the below data

HR,Human Resources
IT,Information Technology

Step 7: Create Pig Script

nano scripts.pig => To open pig scripts file

1.To load employees file:

```
emp = LOAD 'employees.txt' USING PigStorage(',')
AS (id:int, name:chararray, age:int, dept:chararray, salary:int);

DUMP emp;      =>To fetch the data on the output screen
```

Output:(1,Anu,25,HR,30000.0)
(2,Akshay,30,IT,50000.0)
(3,Bindu,28,IT,45000.0)
(4,Disha,35,HR,40000.0)

2. To load department file:

```
dept = LOAD 'dept.txt' USING PigStorage(',')
AS (dept:chararray, dept_name:chararray);

DUMP dept;
```

Output:(HR,Human Resources)
(IT,Information Technology)

3. To filter the data by salary

```
high_salary = FILTER emp BY salary > 40000;
DUMP high_salary;    To fetch the data on the output screen
```

Output:(2,Akshay,30,IT,50000.0)
(3,Bindu,28,IT,45000.0)

4.**To group the data by department**

```
grp = GROUP emp BY dept;
DUMP grp;
```

Output: (HR,{(4,Disha,35,HR,40000.0),(1,Anu,25,HR,30000.0)})
(IT,{(3,Bindu,28,IT,45000.0),(2,Akshay,30,IT,50000.0)})

5. To Project the data

```
proj = FOREACH  emp  GENERATE name, salary;
DUMP proj;                    =>Like Select Statement in Sql
```

Output: (Anu,30000.0)
(Akshay,50000.0)
(Bindu,45000.0)
(Disha,40000.0)

6. **To count the no of employees in each department**

```
dept_count = FOREACH grp GENERATE group,COUNT(EMPLOYEES);
DUMP dept_count;
```

Output:(HR,2)
(IT,2)

7.**To sort the employees in ascending /descending order**

```
sorted_emp = ORDER emp BY salary DESC;
DUMP sorted_emp;
```

Output:(2,Akshay,30,IT,50000.0)
(3,Bindu,28,IT,45000.0)
(4,Disha,35,HR,40000.0)
(1,Anu,25,HR,30000.0)

8. To fetch the employees by each department(we use joins)

```
joined = JOIN emp BY dept, dept BY dept;
DUMP sorted_emp;
```

Output: (3,Bindu,28,IT,45000.0,IT,Information Technology)
(2,Akshay,30,IT,50000.0,IT,Information Technology)

Step 8: To Run Script

```
pig -x local scripts.pig
```

Program 7: Use Hive to create, alter and drop database, table, views

Step-1: To Install PySpark

Type the below command

!pip install pyspark

Step-2: To Start Spark Session with Hive Support

Open the file:

nano hive.py

```
from pyspark.sql
import SparkSession spark = SparkSession.builder \
.appName("HiveSimulation") \
.enableHiveSupport() \
.getOrCreate()
print("Spark Session Created")
```

In hive.py file:

a: To Create Database

```
spark.sql("CREATE DATABASE IF NOT EXISTS mydb")
spark.sql("USE mydb") print("Database created and selected")
```

b: To Create Table

```
spark.sql(""" CREATE TABLE IF NOT EXISTS student(
        id INT,
        name STRING
)
USING PARQUET
""")
print("Table created")
```

c: To Insert Data

```
spark.sql("INSERT INTO student VALUES (1, 'Riya'), (2, 'Shriya')")
print("Data inserted")
```

d: **To Display data**

```
print("Table Data:")
spark.sql("SELECT * FROM student").show()
```

e**: To Create View**

```
spark.sql("CREATE OR REPLACE VIEW student_view AS
SELECT name FROM student")
print("View created")
```

f**: To view data:**

```
print("View Data:")
spark.sql("SELECT * FROM student_view").show()
```

g**: To Drop Table**

```
spark.sql("DROP TABLE student") print("Table dropped")
```

h**: To drop database**

```
spark.sql("DROP DATABASE mydb       CASCADE")
print("Database dropped")
```

i: **To Alter Table**

```
spark.sql("ALTER TABLE student ADD COLUMNS (age INT)")
print("Column added")
```

Output:

```
Switched to database 'mydb'

Database created and selectedTable created
Data inserted
Table Data:
+---+-------+
| id|   name|
+---+-------+
|  2|Shriyaa|
|  1|   Riya|
+---+-------+
```

View created
View Data:

```
+-------+
|  name|
+-------+
|Shriyaa|
|  Riya|
+-------+
```

Column added
Data inserted
Table dropped
Database dropped

Program 8: Implement a word count program in Hadoop and spark.

Step-1:**Download and Install Apache Spark**

To download apache spark:

wget https://downloads.apache.org/spark/spark-3.5.9/spark-3.5.9-binhadoop3.tgz

To extract:

spark:tar -xvzf spark-3.5.9-bin-hadoop3.tgz

To move:

sudo mv spark-3.5.9-bin-hadoop3 /opt/spark

To Set Environment Variables:

nano ~/.bashrc

Add:

export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:
$SPARK_HOME/sbin export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

Verify:

pyspark

Step-2: **Create directory**:

mkdir spark

cd spark_wordcount

Create Input file

nano input.txt

Sample Data:

Welcome to Big data
Welcome to pyspark
Welcome to hadoop

Step-4: **PySpark WordCount Code**

nano wordcount.py

```
from pyspark import SparkContext

sc = SparkContext("local", "WordCount")
text = sc.textFile("input.txt")

counts = (text
.flatMap(lambda line: line.split())
.map(lambda word: (word, 1))
.reduceByKey(lambda a, b: a + b))
 counts.saveAsTextFile("output")

for word, count in counts.collect():
        print(word, count)
 sc.stop()
```

Step-5. Run the Program

spark-submit wordcount.py

Step-6: View Output

cat output/part-00000

Output: Welcome 3
to 3
Big 1
data 1
pyspark 1
hadoop 1

Program 9: Use CDH(cloudier distribution for Hadoop) and HVE(Hadoop user interface) to analyse data and generate report for sample dataset.

(i) Prepare Sample Datasets
Create sales.csv order_id,customer_name,product,quantity,price

1,Alice,Laptop,1,1200
2,Bob,Mouse,2,25
3,Charlie,Keyboard,1,75
4,David,Laptop,1,1150
5,Eve,Monitor,2,300
6,Frank,Mouse,1,25
7,Grace,Keyboard,2,75
8,Hank,Monitor,1,300

Prepare Sample Datasets
Create sales.csv order_id,customer_name,product,quantity,price
1,Alice,Laptop,1,1200
2,Bob,Mouse,2,25
3,Charlie,Keyboard,1,75
4,David,Laptop,1,1150
5,Eve,Monitor,2,300
6,Frank,Mouse,1,25
7,Grace,Keyboard,2,75
8,Hank,Monitor,1,300

(ii) Upload Data via HUE to HDFS
1. Log in to HUE.
2. Log in to HUE.
3. Navigate to File Browser → Upload.

(iii) Create Hive Tables using HUE Query Editor
Create sales table CREATE TABLE sales (
order_id INT, customer_name STRING, product STRING, quantity INT, price DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','

STORED AS TEXTFILE;

Load data into sales
LOAD DATA INPATH '/user/<your_username>/sales_data/sales.csv' INTO TABLE sales;

Create customers table

CREATE TABLE customers (
customer_name STRING, location STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

Load data into customers LOAD DATA INPATH '/user/<your_username>/sales_data/customers.csv' INTO TABLE customers;

(iv) Perform Data Analysis
Queries Filter: Sales where price > 100
SELECT * FROM sales WHERE price > 100;

Join: Combine sales and customer data
SELECT s.order_id, s.customer_name, c.location, s.product, s.quantity, s.price
FROM sales s
JOIN customers c ON s.customer_name = c.customer_name
GROUP BY c.location;

Sort: Customers with highest total spending SELECT s.customer_name, SUM(s.quantity * s.price) AS total_spent FROM sales s
GROUP BY s.customer_name ORDER BY total_spent DESC;

Generate Reports in HUE
Click on the "Query History" tab after executing queries.
Save or Export the results (CSV/Excel).
Use HUE Dashboard or Notebook features to visualize:
• Bar charts of revenue by location.
• Pie charts of product sales.